

A RISP-Based SoC Framework for Edge Computing with Integrated Cryptographic Accelerators

Thanh-Truong Tran, Duy-Huan Nguyen*, Viet-Hung Nguyen, Duc-Anh Hoang
Embedded Systems Design Department, Viettel Semiconductor Center, Viettel Group, VietNam

*Corresponding author: huannd25@viettel.com.vn

Abstract—The rapid proliferation of IoT edge applications demands compact, energy-efficient SoCs capable of operating under strict constraints. To avoid the hardware overhead of conventional cryptographic ISA or pipeline extensions, this paper presents a RISP-based SoC framework integrating a minimalist single-cycle processor core with dedicated AES, SHA3, and RSA cryptographic engines via an AXI4-Lite interface. Implemented on a Xilinx Zynq-7000 FPGA, the RISP core minimizes control logic to just 1,126 LUTs and 175 FFs. Benchmarked against RV32I and Ibex baselines, the RISP-based SoC reduces total power to 0.407 W (0.298 W dynamic) and drastically cuts cycle counts across all cryptographic workloads. Specifically, it achieves 58.18 Mb/s in AES-128 ECB mode (88 cycles), completes 16-block SHA3-256 in 6,871 cycles, and delivers an energy footprint of 3,265.89 $\mu\text{J/bit}$ along with a throughput-per-watt efficiency of 306.19 $\text{Thpt} \cdot \text{W}^{-1}$ for RSA-2048 verification. The proposed architecture provides an optimal hardware offloading paradigm for energy-constrained secure edge computing.

Index Terms—RISC-V, RISP, Cryptographic Hardware, Secure Edge, RISP-Based SoC.

I. INTRODUCTION

The rapid expansion of edge IoT applications necessitates lightweight, hardware-assisted security mechanisms under strict power, area, and cost constraints [1], while conventional RISC-V instruction-set extensions and tightly coupled coprocessors introduce control complexity and hardware overhead that challenge scalability [1]–[4].

To address these challenges, this paper proposes a minimalist SoC framework centered on a single-cycle Instruction-Subset Processor (RISP) coordinating AES, SHA3, and RSA engines via an AXI4-Lite interface [5], completely decoupling system control from cryptographic computation to eliminate pipeline complexity and optimize silicon resource utilization.

This research makes two primary contributions:

- 1) The RISP methodology is instantiated for memory-mapped cryptographic workloads by deriving an optimized instruction subset tailored to the target firmware.
- 2) The framework is evaluated against a baseline SoC featuring a five-stage pipelined RV32IMB core under identical conditions.

The remainder of this paper is organized as follows: Section II reviews literature on domain-specific processors and cryptographic integration; Section III details the proposed RISP-based SoC architecture and accelerator designs; Section IV presents FPGA implementation results and performance comparisons; and Section V concludes the work and outlines future directions for edge IoT security.

II. RELATED WORK

ISA-level research focuses on scalar cryptographic instructions and custom Instruction Set Extensions (ISE) to reduce program memory and deliver speedups of $1.5\times$ to $8.6\times$ [2]–[4], [6]–[8]. However, integrating cryptographic logic directly into the processor data-path increases control complexity and can negatively impact the core’s critical path and maximum frequency [3], [4].

While coprocessors and stand-alone accelerators (such as RoCC-integrated AES and high-throughput communication crypto engines) achieve massive performance gains [1], [2], [8]–[11], they often retain full-featured pipelined host processors that impose unnecessary resource overhead for extreme-edge controllers [6], [9], [11], [12].

Memory-mapped integration via AXI4-Lite or AXI-Stream allows modular, independent accelerator verification [12]–[14], while Instruction-Subset Processor (RISP) methodologies generate application-specific single-cycle cores that reduce area by 8% to 43% and power by 3% to 30% compared to full-ISA processors [5], providing the foundation for our pairing of a minimalist RISP host with memory-mapped AES, SHA3, and RSA accelerators.

III. PROPOSED RISP-BASED SOC ARCHITECTURE

The proposed SoC pairs a minimalist RISP host with AES, SHA3, and RSA engines via an interconnect, decoupling control from security logic for edge efficiency [5].

A. System Overview and Memory-Mapped Integration

The SoC utilizes an AXI4-Lite interconnect for low-latency communication between the RISP host and memory-mapped cryptographic accelerators [9], offloading intensive computations to reduce host duty cycles [11]. Fig. 1 illustrates the architectural framework, featuring the main CPU host coordinated via an interconnect with AES, SHA3, and RSA accelerators, an on-chip memory hierarchy, standard I/O interfaces [15], and a DMA controller.

B. Minimalist RISP Host Core

As illustrated in Fig. 1, the single-cycle, non-pipelined RISP host executes only the instruction subset required for firmware, minimizing pipeline overhead, TCB, and logic resources to provide a verified deterministic and compact root-of-control foundation [5].

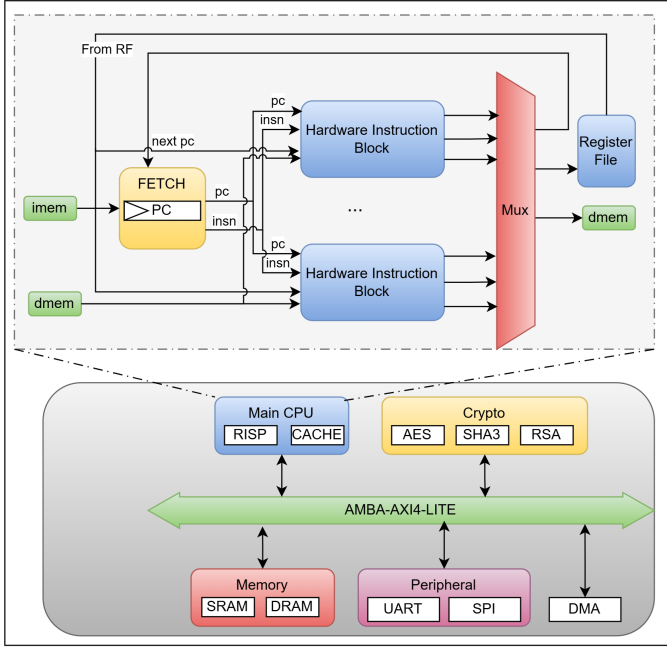


Fig. 1. Overall architecture of the RISP-based SoC and internal processor microarchitecture.

TABLE I
INSTRUCTIONS RETAINED AND OMITTED IN THE RISP HOST

Type	Retained Instructions	Omitted Instruction
B-type	BEQ, BNE, BLT, BGE, BLTU, BGEU	—
R-type	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND	—
I-type	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI	—
Loads	LB, LW, LBU	LH, LHU
Stores	SB, SH, SW	—
U-Types	LUI, AUIPC	—
Jump	JAL, JALR	—

Table I details the workload-specific RISP instructions grouped by format, satisfying the set-completeness criterion:

$$\mathcal{I}_{req} \subseteq \mathcal{I}_{impl} \quad \text{and} \quad \mathcal{I}_{omit} \cap \mathcal{I}_{req} = \emptyset, \quad (1)$$

where $\mathcal{I}_{req}$, $\mathcal{I}_{impl}$, and $\mathcal{I}_{omit}$ denote required, implemented, and omitted instructions, respectively. By retaining essential instructions that prioritize word-aligned access and integer arithmetic, while omitting redundant opcodes, the design significantly reduces sequential logic and control complexity compared to general-purpose baselines.

C. AES Accelerator Architecture

To ensure confidentiality with minimal area, the rolled AES engine employs composite field S-boxes and interfaces with the RISP host via 128-bit memory-mapped registers [4], [12]. As depicted in Fig. 2, memory-mapped CSRs and staging registers manage inputs before mode logic routes blocks through cipher modes (ECB, CBC, CTR, CFB) into the core

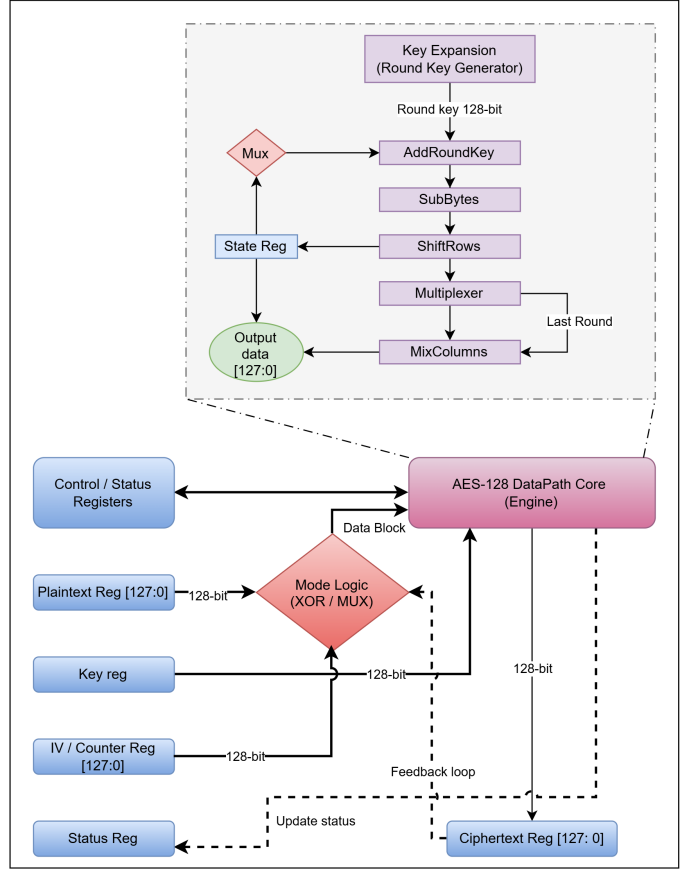


Fig. 2. Block diagram of the AES-128 datapath core and top-level control logic.

engine, concluding with status flags and ciphertext storage [12].

Let $\mathcal{E}_K(\cdot)$ denote the 128-bit AES encryption under key K , where P_i and C_i are the i -th plaintext and ciphertext blocks ($i \geq 1$). Let IV be the initialization vector, T_i the counter state, and $\oplus$ the bitwise XOR. The formulas for the four supported modes are:

$$C_i = \mathcal{E}_K(P_i) \quad (\text{ECB}), \quad (2)$$

$$C_i = \mathcal{E}_K(P_i \oplus C_{i-1}), \quad C_0 = IV \quad (\text{CBC}), \quad (3)$$

$$C_i = P_i \oplus \mathcal{E}_K(T_i), \quad T_i = T_0 + i - 1 \quad (\text{CTR}), \quad (4)$$

$$C_i = P_i \oplus \mathcal{E}_K(C_{i-1}), \quad C_0 = IV \quad (\text{CFB}). \quad (5)$$

D. SHA3 Accelerator Architecture

The multimode SHA3 engine provides integrity via a sponge construction and configurable buffers [16]. As illustrated in Fig. 3, an input controller handles handshaking [16], a paddor formats 1088-bit blocks [11], and an internal $F_{\text{permutation}}$ engine executes 24Keccak-f rounds over a 1600-bit state before output extraction [11], [16].

Let M be the input message, padded into blocks m_i of size $r = 1088$ bits. The internal state S is initialized to a 1600-bit

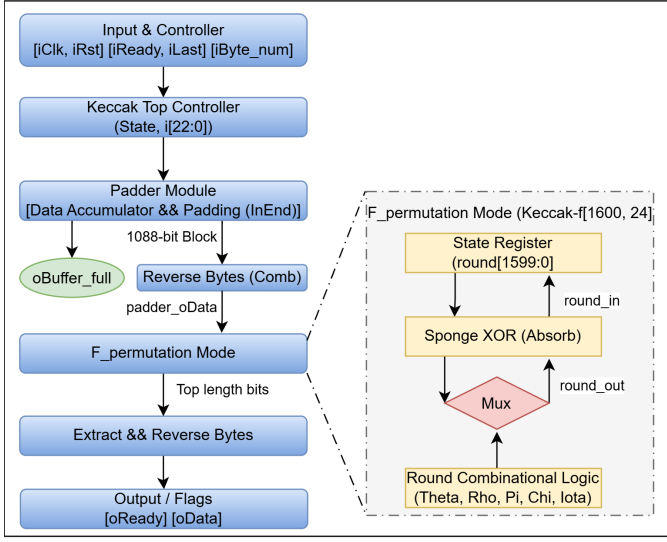


Fig. 3. High-level architecture and F-permutation processing pipeline of the SHA3 accelerator.

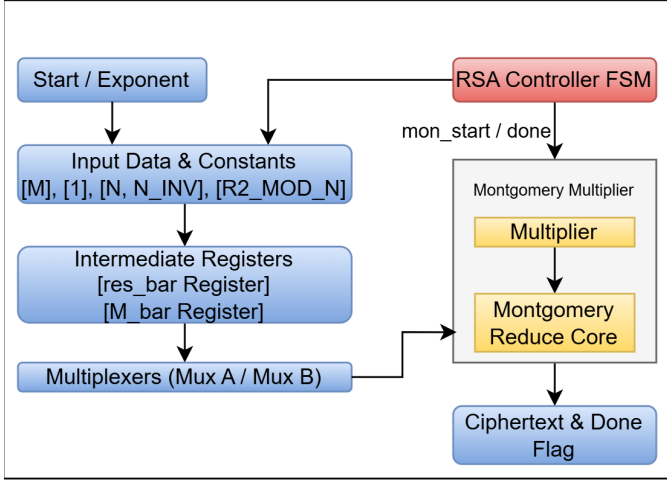


Fig. 4. RSA cryptographic accelerator architecture featuring a Montgomery multiplication core.

zero array. The sponge operation and 5-step round permutation $(\theta, \rho, \pi, \chi, \iota)$ are formulated as:

$$\text{Padding: } S^{(0)} = 0^{1600}, \quad m_i = \text{Pad}(M) \quad (6)$$

$$\text{Absorbing: } S^{(i)} = \mathcal{F} \left(S^{(i-1)} \oplus (m_i \parallel 0^{1600-r}) \right) \quad (7)$$

$$\text{Permutation: } \mathcal{F} = \iota \circ \chi \circ \pi \circ \rho \circ \theta \quad (8)$$

$$\text{Squeezing: } Z_i = \text{Truncate}_d(S^{(i)}) \quad (9)$$

E. RSA-Oriented Modular Arithmetic Engine

A dedicated modular arithmetic engine supports RSA and ECC primitives using a word-serial Montgomery multiplier for compact, high-precision finite field calculations [1], decoupling long-latency modular exponentiation from the host RISP to mitigate side-channel vulnerabilities and ensure consistent

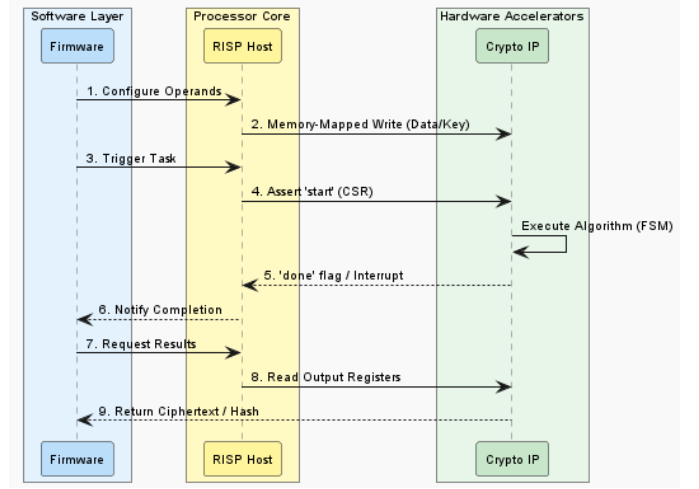


Fig. 5. Control and data exchange protocol between software and memory-mapped cryptographic hardware.

performance. As illustrated in Fig. 4, an FSM controller and input staging module coordinate parameters—such as plaintext M , modulus N , and Montgomery constants—routing them via multiplexers to execute modular exponentiation [1].

A modular arithmetic engine accelerates RSA/ECC primitives via a word-serial Montgomery multiplier [1]. As shown in Fig. 4, an FSM controller manages staging registers to process operands (M, N) and execute exponentiation [1].

$$\text{Domain Conversion: } \bar{A} = A \cdot R \pmod{N} \quad (10)$$

$$\text{Montgomery Mul: } \text{MonMul}(\bar{A}, \bar{B}) = \bar{A}\bar{B}R^{-1} \pmod{N} \quad (11)$$

$$\text{Exponentiation: } C = M^d \pmod{N} \quad (12)$$

F. Hardware-Software Control Flow

A lightweight register interface coordinates firmware and accelerators [11]. As detailed in Fig. 5, the firmware requests operations, and the RISP host writes operands via memory-mapped transactions before asserting a CSR 'start' trigger to initiate hardware execution [9], [12]. Upon completion, a 'done' flag or interrupt notifies the host to retrieve and return results to the application layer [9].

IV. EXPERIMENTAL EVALUATION AND RESULTS

This section evaluates the RISP-based SoC against RV32I and Ibex baselines on a Xilinx Zynq-7000 FPGA at 40 MHz via AXI4-Lite. Fig. 6 illustrates the SoC block diagram, featuring the RISP host integrated with AES, SHA3, and RSA accelerators via an AXI interconnect.

A. System-Level Resource Utilization

As summarized in Table II, the proposed RISP-based SoC achieves a significantly compact and energy-efficient hardware footprint compared to standard RV32I and Ibex baselines, making it exceptionally well-suited for resource-constrained edge IoT devices. Specifically, the minimalist RISP core

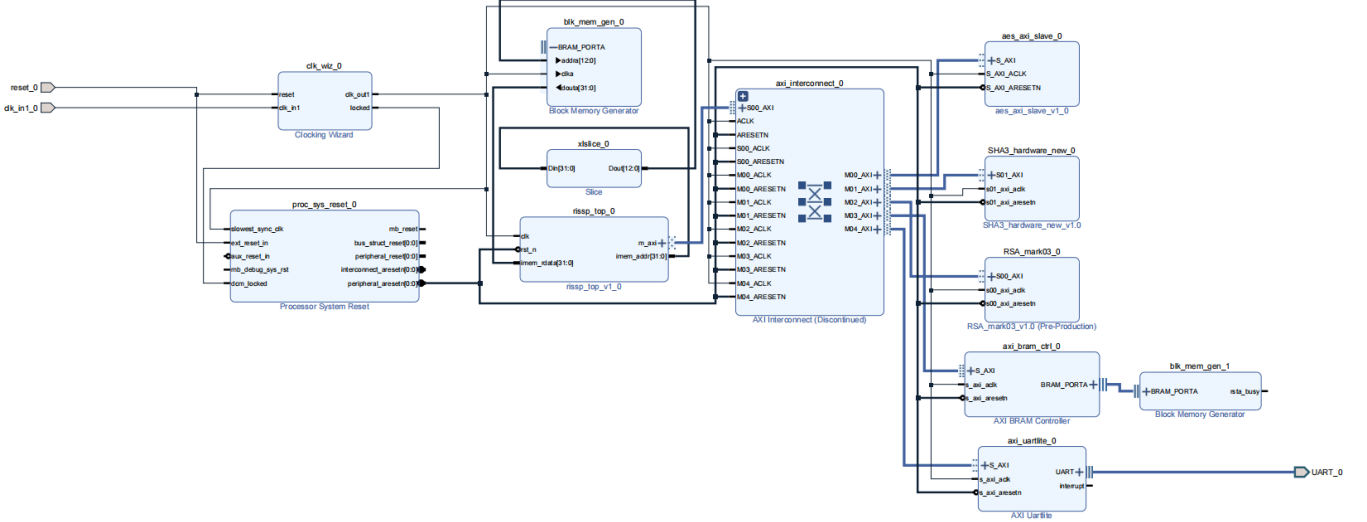


Fig. 6. Vivado block diagram of the proposed SoC architecture.

TABLE II
HARDWARE RESOURCE AND PERFORMANCE METRICS

Metric	RISP-Based SoC	RV32I-Based SoC [17]	Ibex-Based SoC [18]
CLB LUTs	17,180	17,632	18,484
CLB FFs	24,748	26,233	26,401
DSP Slices	4	4	4
Core LUTs	1,126	1,584	2,437
Core FFs	175	1,663	1,841
Dyn. Power (W)	0.298	0.328	0.336
Stat. Power (W)	0.109	0.110	0.110
Total Power (W)	0.407	0.438	0.446
WNS (ns)	5.308	5.249	7.308
Freq. (MHz)	50.78	50.63	56.50

Note: FFs = Flip-Flops; Dyn. = Dynamic; Stat. = Static; WNS = Worst Negative Slack; Freq. = Single-run slack-based frequency estimate.

utilizes merely 1,126 *LUTs* and 175 Flip-Flops, representing a dramatic reduction in core logic complexity relative to the RV32I (1,584 *LUTs*, 1,663 *FFs*) and Ibex (2,437 *LUTs*, 1,841 *FFs*) architectures. Furthermore, the system successfully lowers total power consumption to 0.407W (with a dynamic power of just 0.298W), outperforming both baseline platforms while maintaining a robust operating frequency of 50.78MHz and a positive worst negative slack (WNS) of 5.308ns. These characteristics demonstrate an optimal balance between low-power operation and hardware compactness tailored for secure edge computing environments.

B. Performance Metric Formulations

Systematic evaluation of the execution efficiency and hardware trade-offs for the proposed RISP-based SoC architecture was conducted through comprehensive performance metrics—including execution cycles, data throughput, energy consumption, and energy efficiency—benchmarked against baseline implementations under identical operational conditions.

The execution time of a workload is obtained directly from the measured cycle count and the clock period:

$$t = N_{\text{cycles}} \times T. \quad (13)$$

Building upon this, throughput—expressed in Mb/s (megabits per second)—normalizes the amount of processed data, S_{block} , by the execution time:

$$\text{Throughput} = \frac{S_{\text{block}}}{t} = \frac{S_{\text{block}}}{N_{\text{cycles}} \times T}, \quad (14)$$

where S_{block} is 128 bits for every AES-128 mode (ECB, CBC, CTR, CFB). For SHA3-256, S_{block} represents the actual input-message length in bits (message bytes $\times 8$) rather than the 1088-bit algorithmic rate.

Furthermore, energy consumption is derived from the measured power and execution time, where energy per bit normalizes this figure by the total amount of data processed:

$$\text{Energy} = P \times t = P \times N_{\text{cycles}} \times T, \quad (15)$$

$$\text{Energy/bit} = \frac{\text{Energy}}{S_{\text{block}}}. \quad (16)$$

Energy is reported in nJ for AES-128 and SHA3-256 due to their short execution times, and in μJ for RSA-2048 given its substantially longer execution time.

To evaluate overall energy efficiency for cross-core comparisons, throughput per watt (Mb/s/W) serves as the primary figure of merit:

$$\text{Mb/s/W} = \frac{\text{Throughput (Mb/s)}}{P \text{ (W)}}, \quad (17)$$

this quantity is always the reciprocal of Energy/bit up to a factor of 1000, providing a convenient self-consistency check:

$$\text{Mb/s/W} \approx \frac{1000}{\text{nJ/bit}}. \quad (18)$$

TABLE III
FUNCTIONAL VERIFICATION VECTORS FOR AES-128 MODES

Mode	Input	IV / Counter	Expected Ciphertext	Observed Ciphertext
ECB	00112233	—	69c4e0d8	69c4e0d8
	44556677		6a7b0430	6a7b0430
	8899aabb		d8cdb780	d8cdb780
	ccddeeff		70b4c55a	70b4c55a
CBC	30c81c46	5086cb9b	73bed6b8	73bed6b8
	a35ce411	507219ee	e3c1743b	e3c1743b
	e5fbc119	95db113a	7116e69e	7116e69e
	1a0a52ef	917678b2	22229516	22229516
CTR	30c81c46	f0f1f2f3	5ae4df3e	5ae4df3e
	a35ce411	f4f5f6f7	dbd5d35e	dbd5d35e
	e5fbc119	f8f9fafb	5baf0902	5baf0902
	1a0a52ef	fcfdff01	0db03eab	0db03eab
CFB	f69f2445	26751f67	c04b0535	c04b0535
	df4f9b17	a3cbb140	7c5d1c0e	7c5d1c0e
	ad2b417b	b1808cf1	eac4c66f	eac4c66f
	e66c3710	87a4f4df	9ff7f2e6	9ff7f2e6

Note: Complete 128-bit input, IV, counter, and ciphertexts for AES modes.

TABLE IV
FUNCTIONAL VERIFICATION VECTORS FOR SHA3-256

N blocks	Message (Bytes)	Message Summary (Hex)	Output Hash (Hex)
1	128	6bc1bee2 2e409f96 e93d7e11 7393172a ...	d1985c30 73f0ff34 c1717893 0e28f04e ...
4	536	6bc1bee2 2e409f96 6bc1bee3 2e409f97 ...	7df3d9cd 8212808e b2a3ac20 38893a49 ...
16	2168	6bc1bee2 2e409f96 6bc1bee3 2e409f97 ...	c14e1d9a 4b135dc8 b88f8272 0d4088bb ...

Note: Variable-length message blocks and output hashes.

TABLE V
FUNCTIONAL VERIFICATION VECTORS FOR RSA-2048

Module	Modulus N (Hex)	Exp E	Input M (Hex)	Output $C = M^E \pmod{N}$ (Hex)
RSA-2048	e49f31a1 ff172739 2d37e0d6 e7f0d8dc ...	65537	4d1a8fd6 6d1ed78c 3f0230f2 bfc14dd0 ...	b3bc622e 5476ecf0 0955af40 215a31d3 ...

Note: 2048-bit modulus, exponent, message, and cipher output vectors.

For asymmetric workloads such as RSA-2048 verification, the operation constitutes a single modular exponentiation rather than a continuous data stream. Consequently, throughput is expressed in verify/s instead of Mb/s:

$$t = N_{\text{cycles}} \times T, \quad (19)$$

$$\text{Energy} = P \times N_{\text{cycles}} \times T, \quad (20)$$

$$\text{Throughput} = \frac{1}{t} \text{ (verify/s)}, \quad (21)$$

$$\text{Throughput/W} = \frac{1/t}{P} \text{ (Thpt} \cdot \text{W}^{-1}\text{)}. \quad (22)$$

C. Functional Verification

To validate the functional correctness of the proposed cryptographic accelerators, extensive verification was conducted using standard NIST test vectors across diverse operational

TABLE VI
COMPREHENSIVE PERFORMANCE AND CYCLE COMPARISON

Engine	Operation / Mode	RISP	RV32I [17]	Ibex [18]
AES-128	ECB (Cycles)	88	93	90
	Non-ECB (Cycles)	124	129	126
	ECB (Mb/s)	58.18	55.05	56.89
	Non-ECB (Mb/s)	41.29	39.69	40.63
	ECB (nJ/bit)	7.00	7.95	7.84
	Non-ECB (nJ/bit)	9.85	11.03	10.97
	ECB (Mb/s/W)	142.95	125.68	127.56
	Non-ECB (Mb/s/W)	101.45	90.62	91.10
SHA3	1-block (Cycles)	492	500	496
	1-block (Mb/s)	83.25	81.92	82.58
	1-block (Mb/s/W)	204.55	187.03	185.16
	4-block (Cycles)	1,771	1,911	1,907
	4-block (Mb/s)	96.85	89.75	89.93
	4-block (Mb/s/W)	237.96	204.91	201.64
	16-block (Cycles)	6,871	7,419	7,415
	16-block (Mb/s)	100.97	93.51	93.56
	16-block (Mb/s/W)	248.08	213.49	209.78
RSA-2048	Verify (Cycles)	320,972	321,039	320,974
	Energy ($\mu\text{J/bit}$)	3,265.89	3,515.38	3,578.86
	Throughput (verify/s)	124.62	124.60	124.62
	Throughput (Thpt $\cdot \text{W}^{-1}$)	306.19	284.47	279.42

Note: ECB = Electronic Codebook mode; Non-ECB = CBC/CTR/CFB modes; AES cycles are evaluated per 128-bit data block; SHA-3 block payloads correspond to 128 B (1-block), 536 B (4-block), and 2,168 B (16-block); Mb/s = Megabits per second; nJ/bit = Nanojoules per bit; $\mu\text{J/bit}$ = Microjoules per bit; Mb/s/W and $\text{Thpt} \cdot \text{W}^{-1}$ = Throughput-per-watt metrics.

modes and message payload lengths. As detailed in Table III, the AES-128 hardware engine successfully processes 128-bit blocks across Electronic Codebook (ECB), Cipher Block Chaining (CBC), Counter (CTR), and Cipher Feedback (CFB) modes, with all observed ciphertexts perfectly matching the expected theoretical values.

Complementing the symmetric evaluation, the SHA3-256 accelerator was thoroughly tested for hash generation capabilities. As presented in Table IV, the core efficiently handles variable-length message loads ranging from 1 to 16 blocks (128 to 2,168 bytes), successfully producing fully compliant cryptographic hash outputs.

Furthermore, to validate the asymmetric cryptographic capabilities, evaluations were performed using standard test vectors for the public-key engine. As illustrated in Table V, the functional verification vectors demonstrate proper execution of the RSA-2048 module, confirming accurate modular exponentiation ($C = M^E \pmod{N}$) under standard 2048-bit modulus, exponent, and message inputs.

D. Cycle Count and Detailed Performance Comparison

Systematic evaluation of the execution efficiency and hardware trade-offs for the proposed RISP-based SoC architecture was conducted through comprehensive performance metrics—including execution cycles, data throughput, energy consumption, and energy efficiency—benchmarked against baseline implementations under identical operational conditions. Table VI summarizes the execution cycles, throughput, and energy metrics across the RISP-based SoC architecture and the RV32I and Ibex baseline SoCs for AES-128, SHA3-256, and RSA-2048 workloads. For symmetric AES-128 operations,

the RISP architecture reduces the execution overhead to 88 cycles per block in ECB mode and 124 cycles in non-ECB modes, outperforming both baseline processors while delivering higher throughput (58.18 Mb/s for ECB and 41.29 Mb/s for non-ECB) and lower energy consumption (7.00 nJ/bit and 9.85 nJ/bit respectively). In the hash-based SHA3-256 workloads, the custom data path maintains superior cycle efficiency across 1-block (492 cycles), 4-block (1,771 cycles), and 16-block (6,871 cycles) message loads—completing a 16-block payload significantly faster than the 7,419 cycles required for RV32I and 7,415 cycles for Ibex, yielding up to 100.97 Mb/s throughput and an exceptional efficiency of 248.08 Mb/s/W. Finally, for the asymmetric RSA-2048 verification, all architectures require comparable pure cycle counts (320,972 cycles for RISP versus 321,039 for RV32I) due to the inherent algorithmic complexity of modular exponentiation. However, RISP achieves a superior energy footprint of 3,265.89 μ J/bit and an enhanced throughput-per-watt metric of 306.19 Thpt $\cdot$ W⁻¹.

The experimental results highlight a profound performance-energy trade-off inherent to the RISP design philosophy. While general-purpose RISC-V cores like RV32I and Ibex rely on software-driven instruction sequences that incur high instruction fetch overhead and extended register file pressure, RISP leverages dedicated hardware accelerators and optimized instruction subsets to minimize active execution time. This reduction in execution cycles directly shrinks the active energy integration window ($E = P \cdot T_{\text{exec}}$), yielding substantial gains in energy efficiency (Mb/s/W) without triggering timing violations. Although complex operations like RSA-2048 are bounded by intensive arithmetic operations where cycle count reductions diminish, the streamlined control logic of RISP successfully prevents idle power leakage, confirming that custom accelerator integration provides an optimal balance between high data throughput and ultra-low energy consumption for resource-constrained embedded systems.

V. CONCLUSION

In this work, a secure and modular SoC architecture for extreme-edge IoT was developed, featuring a single-cycle RISP host coupled with AES, SHA3, and RSA accelerators via AXI4-Lite. The decoupling of control from cryptographic operations ensured a deterministic root-of-control alongside a minimal hardware overhead. FPGA evaluations demonstrated significant resource and power savings over general-purpose alternatives, validating the effectiveness of the proposed hardware-accelerated security framework. Future work will focus on hardening the hardware framework against physical side-channel attacks (SCA), integrating countermeasures for secure cryptographic execution in untrusted environments, and extending support for emerging post-quantum cryptographic primitives.

REFERENCES

- [1] W. Wang, J. Han, X. Cheng, and X. Zeng, "An energy-efficient crypto-extension design for risc-v," *Microelectronics Journal*, vol. 115, p. 105165, 2021.
- [2] D. T. Pham, T. Van Luu, D. T. Tran, N. H. Dao, D. V. T. Le, T. Van Nguyen, T. M. Pham, and N. X. Pham, "Sha-rv: A risc-v accelerator for sha-224/256 with cycle reduced isa extensions for blockchain applications," *REV Journal on Electronics and Communications*, vol. 15, no. 4, 2026.
- [3] C. Pham-Quoc and N. BINH, "Secure and energy-efficient edge computing platform with customized risc-v," *Journal of Advances in Information Technology*, vol. 15, no. 12, pp. 1392–1399, 2024.
- [4] T. M. Ignatius, T. B. Singha, and R. P. Palathinkal, "Power side-channel attacks on crypto-core based on risc-v isa for high-security applications," *IEEE Access*, vol. 12, pp. 150 230–150 248, Oct. 2024.
- [5] A. Raisiardi, K. Jordanou, J. Kufel, K. Gudimetla, K. Myny, and E. Ozer, "Flexing RISC-V instruction subset processors to extreme edge," in *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, Oct. 2025, pp. 1147–1159.
- [6] H. Cheng, J. Großschädl, B. Marshall, D. Page, and T. Pham, "Risc-v instruction set extensions for lightweight symmetric cryptography," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 193–237, Nov. 2023.
- [7] B. Marshall, G. R. Newell, D. Page, M.-J. O. Saarinen, and C. Wolf, "The design of scalar aes instruction set extensions for risc-v," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 109–136, 2021.
- [8] G. Nişancı, P. G. Flikkema, and T. Yalçın, "Symmetric cryptography on risc-v: Performance evaluation of standardized algorithms," *Cryptography*, vol. 6, no. 3, p. 41, 2022.
- [9] T. Gomes, P. Sousa, M. Silva, M. Ekpanyapong, and S. Pinto, "Fac-v: An fpga-based aes coprocessor for risc-v," *Journal of Low Power Electronics and Applications*, vol. 12, no. 4, p. 50, 2022.
- [10] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocci, S. Saponara, L. Fanucci *et al.*, "A risc-v post quantum cryptography instruction set extension for number theoretic transform to speed-up crystals algorithms," *IEEE ACCESS*, vol. 9, pp. 150 798–150 808, 2021.
- [11] K.-M. Ma, D.-H. Le, C.-K. Pham, and T.-T. Hoang, "Design of an soc based on 32-bit risc-v processor with low-latency lightweight cryptographic cores in fpga," *Future Internet*, vol. 15, no. 5, p. 186, May 2023.
- [12] O. Simola, A. Korsman, V. Hirvonen, A. Tarkka, J. Helander, K. Jarvinen, M. Kosunen, and J. Ryyanen, "Risc-v core with aes-256 accelerator," 2024.
- [13] C.-T. Axinte, A. Stan, and V.-I. Manta, "Embedded streaming hardware accelerators interconnect architectures and latency evaluation," *Electronics*, vol. 14, no. 8, p. 1513, 2025.
- [14] P. Nasahl, R. Schilling, M. Werner, and S. Mangard, "Hector-v: A heterogeneous cpu architecture for a secure risc-v execution environment," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, 2021, pp. 187–199.
- [15] T.-T. Hoang, C. Duran, K.-D. Nguyen, T.-K. Dang, Q. N. Q. Nhu, P. H. Than, X.-T. Tran, D.-H. Le, A. Tsukamoto, K. Suzuki *et al.*, "Low-power high-performance 32-bit risc-v microcontroller on 65-nm silicon-on-thin-box (sotb)," *IEICE Electronics Express*, vol. 17, no. 20, pp. 20 200 282–20 200 282, 2020.
- [16] H.-T. Huynh, T.-P. Dang, T.-K. Tran, T.-T. Hoang, and C.-K. Pham, "A multimode sha-3 accelerator based on risc-v system," *IEICE Electronics Express*, vol. 21, no. 11, pp. 20 240 156–20 240 156, 2024.
- [17] S. Harris and D. Harris, *Digital design and computer architecture, RISC-V edition*. Morgan Kaufmann, 2021.
- [18] N. Gallmann, P. Vogel, P. D. Schiavone, and L. Benini, "From swift to mighty: A cost-benefit analysis of ibex and cv32e40p regarding application performance, power and area," in *5th Workshop Comput. Archit. Res. With RISC-V (CARRV)*, 2021.